BookMyShow - Movie Ticket Booking System Design

Complete Interview Guide with Conversational Scripts

Print Settings: Landscape mode, monospace font (Courier New/Consolas 9-10pt), narrow margins

---

TABLE OF CONTENTS

1. Requirements & Capacity Estimation
2. High-Level Architecture
3. Database Design
4. Core Workflows
5. Concurrency & Seat Locking
6. Scalability & Optimizations
7. Interview Q&A

SECTION 1: REQUIREMENTS & CAPACITY ESTIMATION

Phase 1: Opening the Conversation (2 minutes)

Interviewer: "Design BookMyShow - a movie ticket booking system."

You: "Great! Before I jump into the design, let me make sure I understand the problem correctly. BookMyShow is like Fandango or AMC - users can search for movies, browse theaters, select seats, and book tickets online. Is that right?"

Interviewer: "Yes, exactly."

You: "Perfect. Let me start by clarifying the functional and non-functional requirements. I want to make sure we're aligned on what's in scope and what's not."

1.1 Functional Requirements

You: "For functional requirements, I'm thinking about two main user personas - regular users and theater administrators."

You: "For regular users, they should be able to:"

- "Search for movies by city, theater name, or movie title"
- "Browse available shows with dates and times"
- "View the actual seat layout of the theater"
- "Select specific seats with real-time availability"
- "Make payments through multiple gateways like cards, UPI, wallets"
- "View their booking history"
- "Cancel bookings with refunds based on timing"
- "Maybe rate and review movies after watching"

You: "For theater admins:"

- "Add and update movies, shows, theaters"
- "Define seat layouts and dynamic pricing"
- "View booking analytics and revenue"

You: "For this interview, should I focus on the core booking flow, or do you want me to cover food ordering and loyalty programs as well?"

Interviewer: "Focus on the core booking flow. Keep food ordering and loyalty out of scope."

You: "Got it. So the critical path is: search → select show → pick seats → payment → confirmation."

1.2 Non-Functional Requirements

You: "Now for non-functional requirements, this is where it gets interesting because BookMyShow has some unique challenges."

You: "First, let me think about scale. India has about 100 million active users, and let's say there are around 10 million bookings per month. Does that sound reasonable?"

Interviewer: "Yes, that's in the ballpark."

You: "Great. So scale-wise we're looking at high read traffic with moderate writes. Now the critical NFRs I'm thinking about:"

You: "**Availability** - This needs to be very high, maybe 99.99%, because booking windows are time-sensitive. If the system is down on Friday evening when a big movie releases, that's lost revenue."

You: "**Latency** - Search should be fast, under 500 milliseconds. But booking completion can be slightly slower, maybe under 2 seconds, because payment gateways are involved."

You: "**Consistency** - Here's the big one: seat booking MUST have strong consistency. We absolutely cannot double-book a seat. Even one double-booking will cause a major incident at the theater. So this is ACID territory."

You: "On the flip side, search results and reviews can have eventual consistency. If a review takes 5 seconds to appear, that's acceptable."

You: "**Concurrency** - The system must handle massive concurrent access, especially during popular movie releases. Think 10,000 users trying to book the same show simultaneously."

You: "**Fairness** - First-come-first-served for seat selection. We need to avoid scenarios where someone selects a seat but another user gets it."

You: "Does this align with your expectations, or should I adjust any of these?"

Interviewer: "That's good. Now show me the capacity estimation."

1.3 Capacity Estimation

You: "Let me work through the numbers on the board. I'll start with the assumptions we agreed on."

💡 **MUST WRITE ON BOARD:**

```
CAPACITY ESTIMATION

Given:
- 100M registered users
- 10M bookings/month
- Avg 2.5 tickets/booking
- 80% bookings on weekends (Friday-Sunday)

Daily Traffic:
─────────────────────────────────────────────────
Average day:  10M / 30 = 333K bookings/day
Peak day:     333K × 1.6 = 533K bookings/day

Bookings per second:
─────────────────────────────────────────────────
Average:      333K / 86400s = 3.85 bookings/sec
Peak:         533K / (8 hours × 3600s) = 18.5 bookings/sec
Surge:        18.5 × 5 = 92 bookings/sec (Friday 7-9 PM)

Read/Write:   100:1 (users browse 100 shows before booking 1)
Search QPS:   385 QPS average, 9,200 QPS peak

Storage:
─────────────────────────────────────────────────
Per booking:  2 KB (metadata + indexes)
Per ticket:   500 bytes
Daily:        (333K × 2KB) + (833K × 500B) = 1.08 GB/day
Annual:       ~395 GB/year
5-year:       2 TB (raw) → 6 TB (with replication 3×)

Bandwidth:
─────────────────────────────────────────────────
Incoming:     ~500 KB/s (booking data + movie posters)
Outgoing:     ~50 MB/s (search results + images + videos)
```

You: "So the key takeaway is we're dealing with high read traffic, moderate writes, and bursty peak loads. The 92 bookings per second during peak is the number we need to design for."

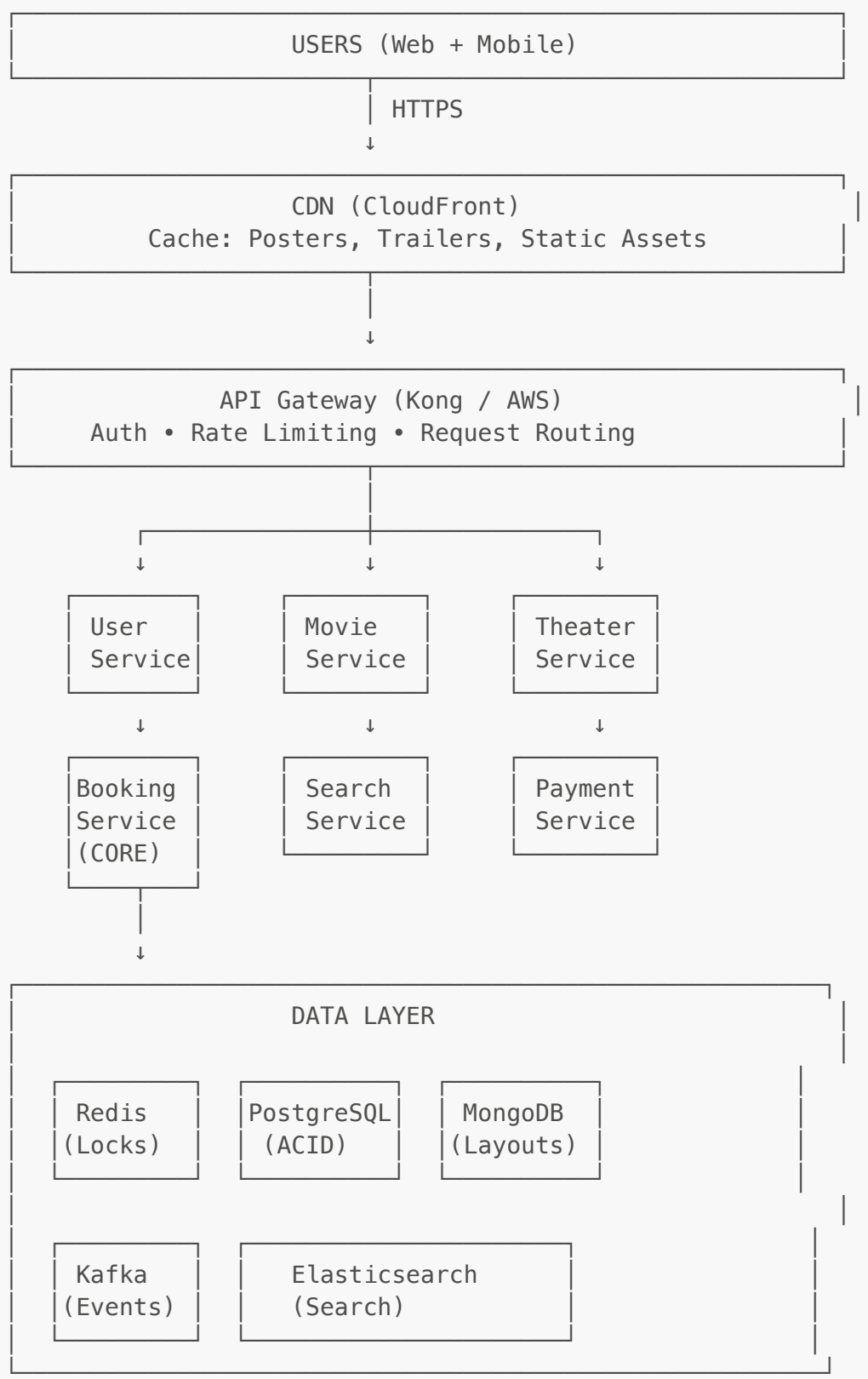
SECTION 2: HIGH-LEVEL ARCHITECTURE

Phase 2: Architecture Discussion (10 minutes)

You: "Now let me sketch out the high-level architecture. I'm going to use a microservices approach because different components have different scaling needs."

You: "Let me draw this layer by layer, starting from the user all the way to the data stores."

💡 **MUST DRAW ON BOARD:**



You: "Let me explain each layer and why I chose these components:"

You: "**CDN Layer** - CloudFront or Cloudflare. Movie posters, trailers, and static assets are cached here globally. This reduces load on origin servers and gives us low latency worldwide."

You: "**API Gateway** - Kong or AWS API Gateway. This handles authentication with JWT tokens, rate limiting to prevent abuse, SSL termination, and routes requests to the right microservice."

You: "**Microservices Layer** - I'm breaking this into focused services:"

You: "- **User Service** - Handles authentication, user profiles, booking history" **You:** "- **Movie Service** - CRUD operations for movies, ratings, reviews" **You:** "- **Theater Service** - Manages theaters, screens, and show schedules" **You:** "- **Booking Service** - This is the crown jewel. Handles seat locking, booking creation, and coordination" **You:** "- **Search Service** - Powered by Elasticsearch for fast, geo-based, full-text search" **You:** "- **Payment Service** - Integrates with Stripe, Razorpay, etc. Has circuit breaker for failures"

You: "**Data Layer** - This is a polyglot persistence approach:"

You: "- **Redis** - For distributed seat locking with TTL. This is critical for preventing double bookings" **You:** "- **PostgreSQL** - For booking transactions. We need ACID guarantees here" **You:** "- **MongoDB** - For theater seat layouts because they're highly flexible and nested" **You:** "- **Elasticsearch** - For search with geo-queries and full-text matching" **You:** "- **Kafka** - Event streaming for async operations like notifications"

You: "Does this high-level structure make sense? Any questions before I dive deeper?"

Interviewer: "Why do you need both Redis and database locks?"

You: "Excellent question! This is defense in depth. Redis is the fast path - it fails 99% of concurrent requests in under 10ms without hitting the database. But Redis alone isn't ACID-compliant, so we use database pessimistic locking as a safety net. I'll show you the exact flow when we discuss concurrency."

2.2 Key Design Decisions

You: "Let me quickly highlight the critical design decisions and patterns I'm using:"

You speaking while writing on board:

```
DESIGN PATTERNS & DECISIONS

Pattern                  Why?
_____________________________________________________________
Distributed Locking      Prevent double booking (Redis SETNX)
SAGA Pattern             Handle payment failures gracefully
CQRS                     Read/write separation (replicas)
Event Sourcing           Audit trail via Kafka
Circuit Breaker          Payment gateway resilience
Database Sharding        Scale by city (geographic locality)
```

You: "The distributed locking and SAGA pattern are the most important for BookMyShow because of the concurrent booking challenge."

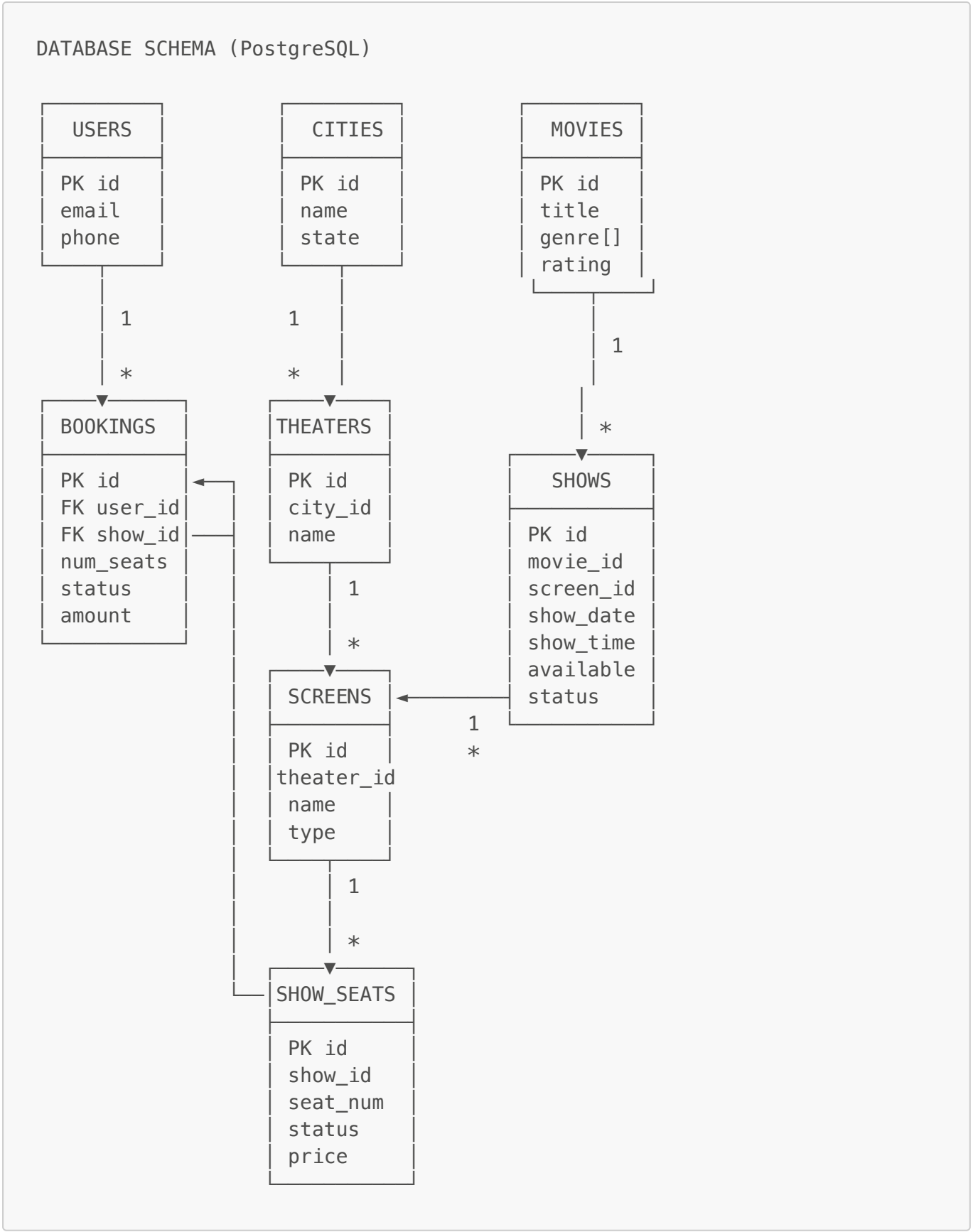
SECTION 3: DATABASE DESIGN

Phase 3: Data Modeling (8 minutes)

You: "Now let me design the database schema. I'll use PostgreSQL for the core transactional data because we need ACID guarantees for bookings."

You: "Let me start with the main entities and their relationships."

💡 **MUST DRAW ON BOARD (Simplified ERD):**



You: "The key relationships are:"

- "Users have many bookings (1:N)"
- "Cities have many theaters (1:N)"
- "Theaters have many screens (1:N)"
- "Movies have many shows (1:N)"
- "Shows belong to screens and movies (N:1)"

- "Bookings link to shows and contain multiple seats"

You: "Now let me detail the critical tables for the booking flow."

You speaking while writing:

💡 **IMPORTANT TABLE SCHEMAS:**

```
-- Core tables for booking flow

CREATE TABLE bookings (
    id BIGSERIAL PRIMARY KEY,
    booking_number VARCHAR(20) UNIQUE NOT NULL,  -- BMS123456789
    user_id BIGINT REFERENCES users(id),
    show_id BIGINT REFERENCES shows(id),
    num_seats INT NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    status VARCHAR(20) NOT NULL,  -- PENDING, CONFIRMED, CANCELLED, EXPIRED
    booked_at TIMESTAMP DEFAULT NOW(),
    expires_at TIMESTAMP,  -- For payment timeout

    INDEX idx_user (user_id),
    INDEX idx_show (show_id),
    INDEX idx_status (status)
);

CREATE TABLE booking_seats (
    id BIGSERIAL PRIMARY KEY,
    booking_id BIGINT REFERENCES bookings(id),
    screen_seat_id BIGINT REFERENCES screen_seats(id),
    seat_number VARCHAR(10),
    price_paid DECIMAL(10,2),

    UNIQUE(booking_id, screen_seat_id)  -- No duplicate seats per booking
);

CREATE TABLE screen_seats (
    id BIGSERIAL PRIMARY KEY,
    screen_id BIGINT REFERENCES screens(id),
    seat_number VARCHAR(10) NOT NULL,  -- A1, B5, etc.
    row_name VARCHAR(5),
    seat_type VARCHAR(20),  -- REGULAR, PREMIUM, VIP, RECLINER
    is_active BOOLEAN DEFAULT true,

    UNIQUE(screen_id, seat_number)  -- No duplicate seat numbers per screen
);

CREATE TABLE payments (
    id BIGSERIAL PRIMARY KEY,
    booking_id BIGINT REFERENCES bookings(id),
    amount DECIMAL(10,2) NOT NULL,
    payment_method VARCHAR(20),  -- CARD, UPI, WALLET
    status VARCHAR(20),  -- PENDING, SUCCESS, FAILED, REFUNDED
```

```
    transaction_id VARCHAR(100),  -- Payment gateway txn ID
    gateway VARCHAR(50),  -- stripe, razorpay
    created_at TIMESTAMP DEFAULT NOW()
);
```

You: "Notice the booking_number is separate from the ID - this is what users see. Status field is critical for state management. The expires_at timestamp is for the 10-minute payment window."

3.2 Redis Data Structures

You: "Now, Redis is doing the heavy lifting for seat locking. Let me show you the key patterns."

You speaking:

💡 **CRITICAL TO UNDERSTAND:**

```
REDIS SEAT LOCKING

Pattern 1: Seat Lock
─────────────────────────────────────────────
Key:    lock:show:{show_id}:seat:{seat_id}
Value:  {user_id}
TTL:    600 seconds (10 minutes)

Command:
SET lock:show:123:seat:A1 user_456 EX 600 NX

Explanation:
- NX = Set only if key doesn't exist (atomic)
- EX 600 = Expire after 10 minutes
- Returns OK if lock acquired, NULL if failed


Pattern 2: Available Seats Cache
─────────────────────────────────────────────
Key:    show:{show_id}:available_seats
Value:  Set of seat IDs
TTL:    30 seconds

Commands:
SADD show:123:available_seats A1 A2 A3 B1 B2
SREM show:123:available_seats A1  (when locked)
SMEMBERS show:123:available_seats  (get all available)
SCARD show:123:available_seats  (count)
```

You: "The NX flag on SET is what makes this atomic. When 1000 users try to book seat A1, Redis executes these sequentially, and only the first one succeeds. This is the foundation of our concurrency handling."

3.3 MongoDB for Seat Layouts

You: "One interesting challenge is that every theater has a different seat layout. PVR Phoenix might have recliners in the front, while INOX has regular seats. Storing this in a fixed PostgreSQL schema is painful."

You: "So I'm using MongoDB for the flexible seat layout definition."

You speaking while writing:

```
// MongoDB Collection: theater_layouts

{
  _id: ObjectId("..."),
  theater_id: "pvr_phoenix_mumbai",
  screen_id: "screen_1",
  layout: {
    total_seats: 120,
    rows: [
      {
        row_name: "A",
        row_type: "RECLINER",  // Row-level classification
        seats: [
          {number: "A1", type: "RECLINER", base_price: 500},
          {number: "A2", type: "RECLINER", base_price: 500},
          {type: "AISLE"},  // Represents gap in seating
          {number: "A3", type: "RECLINER", base_price: 500}
        ]
      },
      {
        row_name: "B",
        row_type: "PREMIUM",
        seats: [
          {number: "B1", type: "PREMIUM", base_price: 350},
          {number: "B2", type: "PREMIUM", base_price: 350}
        ]
      }
    ],
    special_features: ["wheelchair_access", "couple_seats"]
  }
}
```

You: "This gives theaters complete flexibility. They can define custom layouts, add aisles, mark wheelchair seats - all without schema migrations. MongoDB's document model is perfect for this nested structure."

SECTION 4: CORE WORKFLOWS

Phase 4: Deep Dive into Booking Flow (12 minutes)

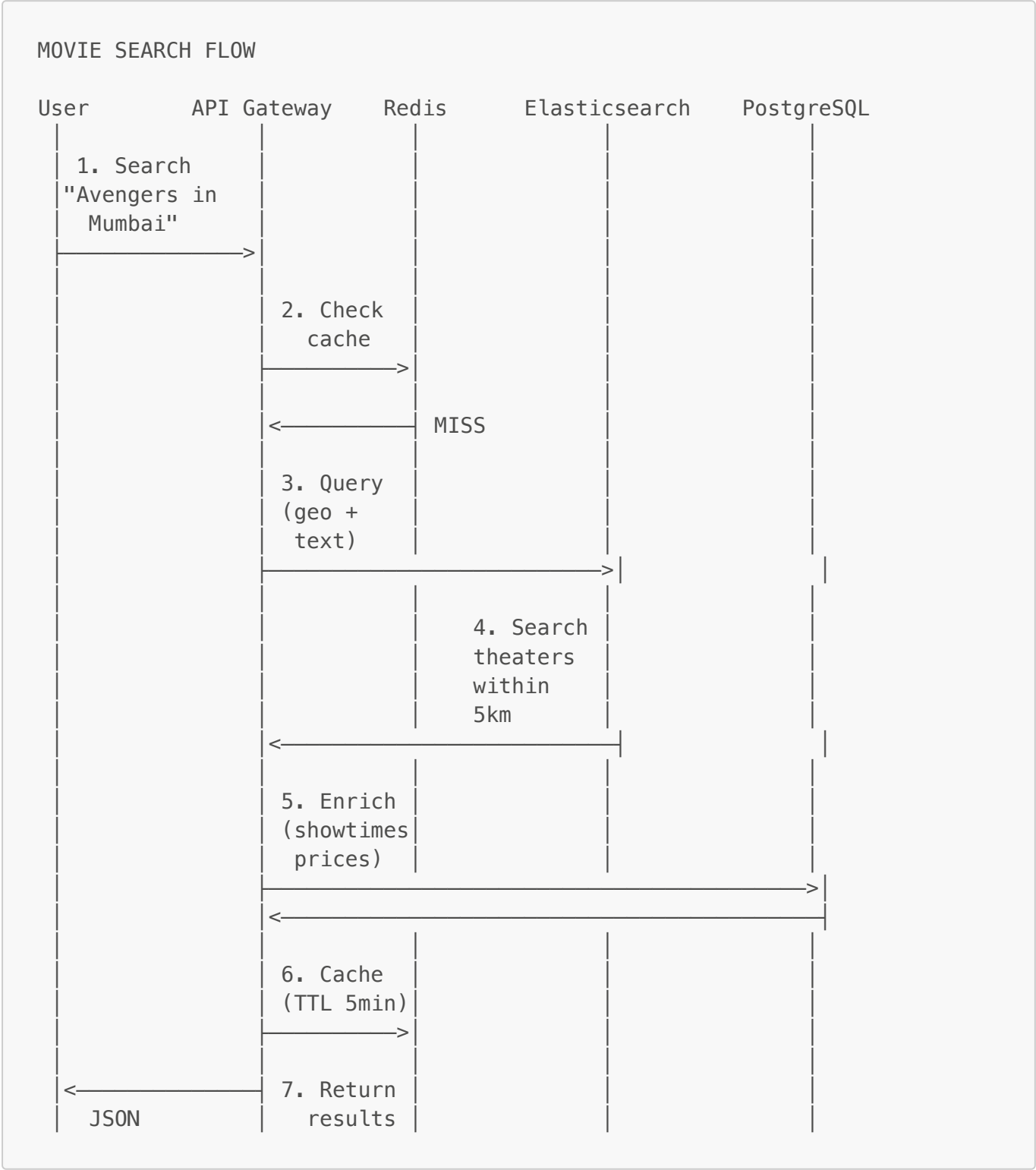
You: "Now let me walk through the two most critical workflows - search and booking. The booking flow is where all the complexity lives."

4.1 Movie Search Flow

You: "Search is relatively straightforward, but I want to show the caching strategy."

You speaking while drawing:

💡 **DRAW SEQUENCE DIAGRAM:**



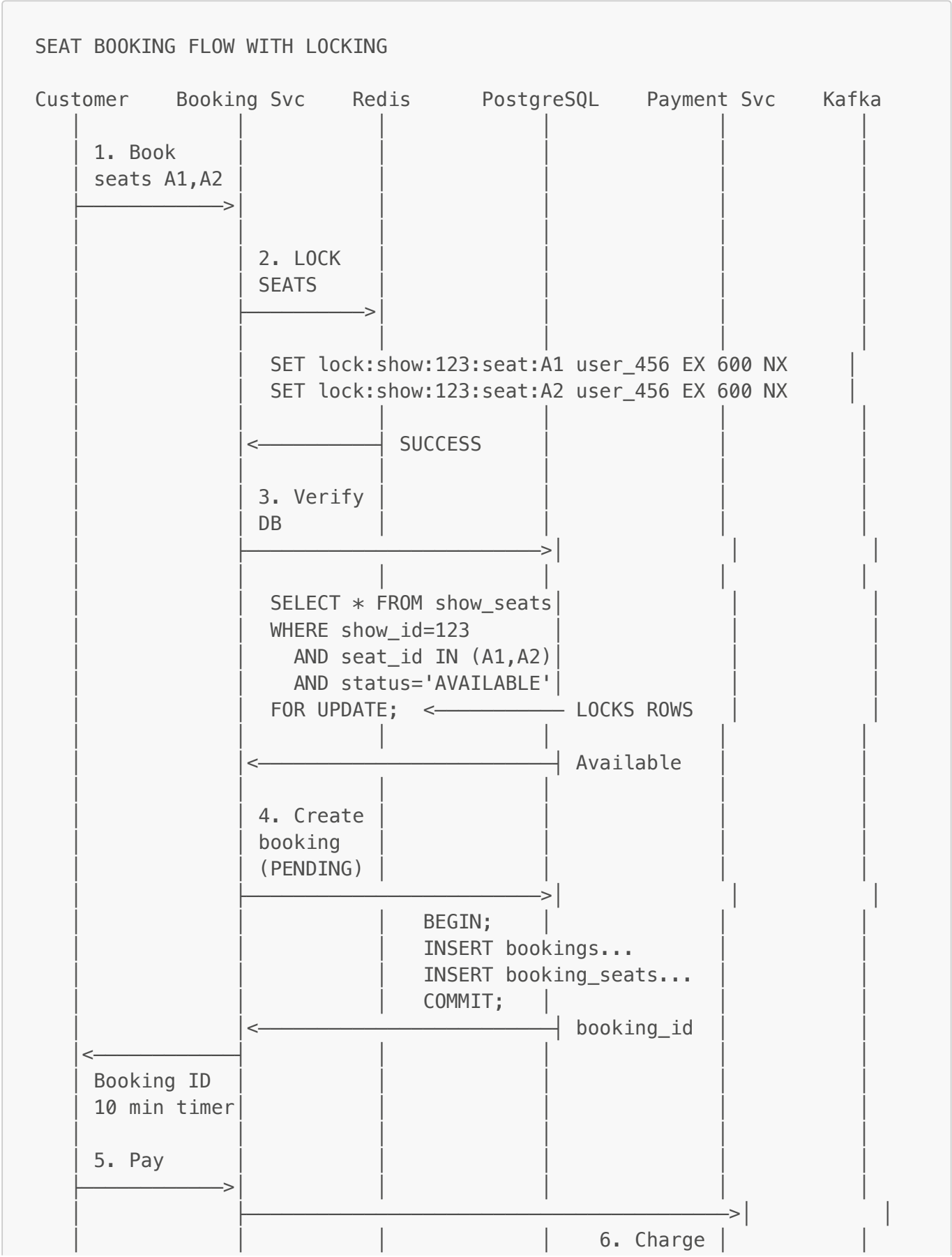
You: "The key optimization here is the 5-minute cache TTL. Search results don't change frequently, so we can serve most requests from Redis without hitting Elasticsearch or PostgreSQL."

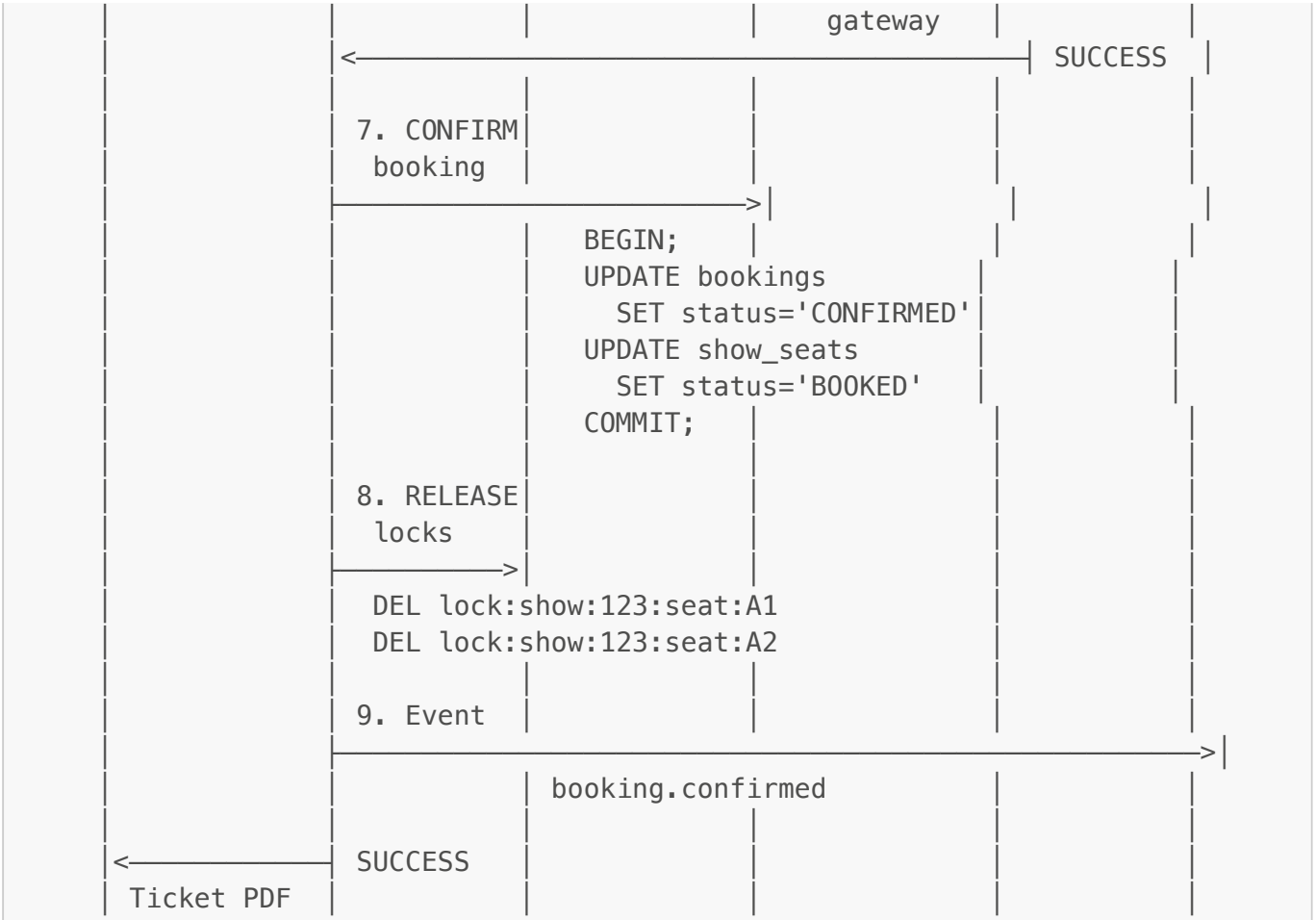
4.2 Seat Booking Flow - THE CRITICAL PATH

You: "Now for the main event - the booking flow with concurrency handling. This is the heart of BookMyShow."

You: "Let me walk you through what happens when a user clicks 'Book' for seats A1 and A2."

💡 **MUST DRAW THIS SEQUENCE:**





You: "Let me explain the critical steps:"

You: "**Step 2 - Redis Locking**: This is atomic. We use SETNX which means 'set if not exists'. If 1000 users try to book seat A1, only one SET succeeds. The others fail immediately with 'already selected' error. This happens in under 10 milliseconds."

You: "**Step 3 - Database Verification**: After getting the Redis lock, we still verify in the database using SELECT FOR UPDATE. This is pessimistic locking - it locks the rows until our transaction commits. This prevents race conditions if Redis fails."

You: "**Step 4 - Pending Booking**: We create the booking with status PENDING. The user now has 10 minutes to complete payment. The seats are locked but not confirmed yet."

You: "**Step 7 - Confirmation**: After successful payment, we update status to CONFIRMED and mark seats as BOOKED in a single atomic transaction."

You: "**Step 8 - Lock Release**: We explicitly delete the Redis locks. But even if this fails, the TTL ensures they expire in 10 minutes."

Interviewer: "What if the payment times out?"

You: "Great question! Let me show you the failure scenarios."

You speaking:

```
FAILURE SCENARIOS

Scenario 1: Lock Acquisition Fails
──────────────────────────────────────────────────
User A: SET lock:show:123:seat:A1 ... → SUCCESS
User B: SET lock:show:123:seat:A1 ... → FAIL

→ User B immediately sees: "Seat already selected"
→ No database query, fails in <10ms


Scenario 2: Lock acquired, DB shows unavailable
──────────────────────────────────────────────────
→ Release Redis lock immediately
→ Return 409: "Seat no longer available"


Scenario 3: Payment timeout (30+ seconds)
──────────────────────────────────────────────────
→ Circuit breaker opens after 30s
→ Return: "Processing payment, we'll notify you"
→ Background job queries payment gateway status
→ If success: confirm booking
→ If failure: cancel booking, release locks


Scenario 4: User takes >10 minutes
──────────────────────────────────────────────────
→ Redis lock expires automatically (TTL)
→ Background cron job finds PENDING bookings
→ Updates status to EXPIRED
→ Seats become available again
```

You: "The beauty of this design is that failures are handled at multiple layers. Redis TTL ensures locks don't stay forever. Background jobs clean up expired bookings. Circuit breakers prevent cascading failures."

4.3 Lock Expiry Background Job

You: "Let me show you the background job that handles lock expiry. This runs every minute."

You speaking:

📝 **WRITE PSEUDO-CODE:**

```
# Background Job - Runs every 1 minute

def expire_old_bookings():
    """
    Cleanup bookings that:
```

```
    1. Status = PENDING
    2. Created >10 minutes ago
    """

    # Find expired bookings
    expired = db.query("""
        SELECT id, show_id
        FROM bookings
        WHERE status = 'PENDING'
          AND created_at < NOW() - INTERVAL '10 minutes'
    """)

    for booking in expired:
        # Update status
        db.execute("""
            UPDATE bookings
            SET status = 'EXPIRED'
            WHERE id = ?
        """, booking.id)

        # Get seats for this booking
        seats = db.query("""
            SELECT screen_seat_id
            FROM booking_seats
            WHERE booking_id = ?
        """, booking.id)

        # Release Redis locks
        for seat in seats:
            key = f"lock:show:{booking.show_id}:seat:{seat.id}"
            redis.delete(key)

        # Publish event
        kafka.publish('booking.expired', {
            'booking_id': booking.id
        })
```

You: "This ensures that even if something goes wrong, seats don't stay locked forever. The TTL on Redis is the first line of defense, this job is the second line."

SECTION 5: CONCURRENCY & SEAT LOCKING

Phase 5: Concurrency Deep Dive (5 minutes)

You: "Let me clearly explain how we handle the concurrency problem. This is THE critical challenge in BookMyShow."

5.1 The Race Condition Problem

You: "Imagine 1000 users clicking 'Book' for seat A1 at the exact same millisecond. Without proper locking, you get double booking."

You speaking while drawing:

💡 **DRAW THIS COMPARISON:**

```
WITHOUT LOCKING (BAD ❌)
─────────────────────────────────────────

Time    User A                  User B
T1      Read seat A1: available
T2                              Read seat A1: available
T3      Check OK ✓
T4                              Check OK ✓
T5      Book seat A1
T6                              Book seat A1
T7      BOTH confirmed → DISASTER! ❌


WITH REDIS LOCKING (GOOD ✓)
─────────────────────────────────────────

Time    User A                  User B
T1      TRY LOCK A1 → SUCCESS ✓
T2                              TRY LOCK A1 → FAIL ❌
T3      Proceed to payment
T4                              Show "Already selected"
T5      Book seat A1
T6      Release lock
T7      ONLY User A gets seat ✓
```

You: "This is why distributed locking is non-negotiable for seat booking systems."

5.2 Lock Implementation

You: "Let me show you the actual lock implementation code. This is important to understand."

You speaking:

💡 **MUST WRITE THIS CODE:**

```java
public class SeatLockService {

    private final RedisTemplate<String, String> redis;
    private static final int LOCK_TTL = 600;  // 10 minutes

    /**
     * Try to acquire locks for all seats atomically
     * If ANY seat fails, rollback all locks
     */
    public boolean lockSeats(Long showId,
                             List<String> seatIds,
                             Long userId) {
```

```
        List<String> lockedSeats = new ArrayList<>();

        try {
            // Try to lock each seat
            for (String seatId : seatIds) {
                String lockKey = String.format(
                    "lock:show:%d:seat:%s", showId, seatId
                );

                // SETNX: Set if Not eXists (atomic operation)
                Boolean acquired = redis.opsForValue()
                    .setIfAbsent(
                        lockKey,
                        userId.toString(),
                        Duration.ofSeconds(LOCK_TTL)
                    );

                if (Boolean.FALSE.equals(acquired)) {
                    // Lock failed! Rollback all acquired locks
                    releaseSeats(showId, lockedSeats);
                    return false;
                }

                lockedSeats.add(seatId);
            }

            return true;  // All seats locked successfully

        } catch (Exception e) {
            // On error, release any acquired locks
            releaseSeats(showId, lockedSeats);
            throw e;
        }
    }

    public void releaseSeats(Long showId, List<String> seatIds) {
        for (String seatId : seatIds) {
            String lockKey = String.format(
                "lock:show:%d:seat:%s", showId, seatId
            );
            redis.delete(lockKey);
        }
    }
}
```

You: "The key insight here is atomicity. The setIfAbsent operation is atomic - Redis processes requests sequentially. So even with 1000 concurrent requests, only one succeeds."

You: "Also notice the rollback logic - if we're booking 3 seats and the third one fails, we release the first two. This prevents partial bookings."

5.3 Database-Level Locking

You: "After Redis lock, we also acquire database lock. Let me show you the SQL."

You speaking:

💡 **IMPORTANT SQL PATTERN:**

```sql
-- Pessimistic locking in PostgreSQL

BEGIN TRANSACTION;

-- Step 1: Lock the rows (FOR UPDATE)
SELECT id, seat_number, status
FROM show_seats
WHERE show_id = 123
  AND seat_id IN ('A1', 'A2')
  AND status = 'AVAILABLE'
FOR UPDATE;

-- This locks the rows. Other transactions wait here.

-- Step 2: If rows returned, seats are available
-- Update the status
UPDATE show_seats
SET status = 'BOOKED',
    booked_by = 456,
    booked_at = NOW()
WHERE show_id = 123
  AND seat_id IN ('A1', 'A2');

-- Step 3: Commit releases the locks
COMMIT;
```

You: "FOR UPDATE is PostgreSQL's pessimistic lock. Any other transaction trying to read these rows with FOR UPDATE will wait until we commit."

You: "So we have two layers:"

- "Redis: Fast, distributed, handles 99% of conflicts in <10ms"
- "Database: ACID, safety net, handles edge cases"

SECTION 6: SCALABILITY & OPTIMIZATIONS

Phase 6: Scaling Discussion (5 minutes)

You: "Now let's talk about how this system scales to handle peak loads."

6.1 Database Sharding

You: "For scaling the database, I'm sharding by city. Here's why this makes sense."

You speaking:

```
DATABASE SHARDING STRATEGY

Shard 0: Mumbai, Pune, Nagpur
Shard 1: Delhi, Gurgaon, Noida
Shard 2: Bangalore, Mysore, Mangalore
Shard 3: Chennai, Coimbatore, Madurai

Routing:
shard_id = hash(city_id) % num_shards

Why City-Based Sharding?
──────────────────────────────────────────────
✓ 95%+ queries filtered by city
✓ Geographic data locality
✓ Hot shard isolation (Mumbai gets dedicated resources)
✓ Easy to add cities (new city = new shard)
✓ No cross-shard joins needed

Example:
User searches "Avengers in Mumbai"
→ Routes to Shard 0
→ All theaters, shows, bookings for Mumbai on one shard
→ Fast, no distributed queries
```

You: "City-based sharding is natural for BookMyShow because users almost never search across cities. You don't book a movie in Mumbai while living in Delhi."

6.2 Caching Strategy

You: "We use multi-level caching to reduce database load."

You speaking:

```
MULTI-LEVEL CACHING

Layer 1: Browser Cache (Client-side)
──────────────────────────────────────────
What:  Movie posters, trailers, CSS, JS
TTL:   24 hours
Why:   Reduces bandwidth, faster page loads


Layer 2: CDN (CloudFront)
──────────────────────────────────────────
What:  Static assets, images, videos
TTL:   24 hours
Why:   Global distribution, low latency
```

```
Layer 3: Redis (Application Cache)
─────────────────────────────────────────────
What:  Search results, theater listings
TTL:   5 minutes (search), 30 sec (seats)
Why:   High throughput, low latency


Layer 4: Database Read Replicas
─────────────────────────────────────────────
What:  Read queries for search, history
Lag:   <1 second (acceptable)
Why:   Offload reads from primary
```

You: "The key is matching cache TTL to data volatility. Search results can be stale for 5 minutes, but seat availability needs 30-second freshness."

6.3 Auto-Scaling

You: "For handling Friday evening surges, we use Kubernetes auto-scaling."

You speaking:

```
KUBERNETES AUTO-SCALING (HPA)

Booking Service:
──────────────────────────────────────────
Min pods:    5
Max pods:    50
Scale up:    CPU > 70% OR requests > 1000/pod
Scale down:  CPU < 30% for 5 minutes

Example Friday 7 PM:
Normal:  5 pods handling 500 req/min
Surge:   Auto-scale to 35 pods for 5000 req/min
Result:  Latency stays <500ms


Database:
──────────────────────────────────────────
Primary:  Vertical scaling (bigger instance)
Replicas: Horizontal scaling (add more replicas)
```

You: "This ensures we handle traffic spikes without over-provisioning during off-peak hours."

6.4 Rate Limiting

You: "To prevent abuse and DDoS, we implement rate limiting at the API gateway."

You speaking:

```
RATE LIMITING (Token Bucket Algorithm)

Per User:
─────────────────────────────────────────
Search:   100 requests/min
Booking:  10 requests/min
Payment:  5 requests/min

Per IP:
─────────────────────────────────────────
500 requests/min (prevents DDoS)

Implementation:
─────────────────────────────────────────
- Redis-based distributed rate limiting
- Key: rate_limit:{user_id}:{endpoint}
- Value: Counter with TTL
- Return 429 Too Many Requests when exceeded
```

You: "Rate limiting ensures fair resource allocation and protects against malicious traffic."

SECTION 7: INTERVIEW Q&A

Common Follow-Up Questions

Interviewer: "How do you prevent double booking?"

You: "We use a two-layer locking mechanism. First, Redis distributed lock with SETNX - this is the fast path that fails 99% of concurrent requests in under 10ms. Second, PostgreSQL pessimistic lock with SELECT FOR UPDATE - this provides ACID guarantees at the database level."

You: "The combination gives us both speed and correctness. Redis handles high concurrency, database ensures consistency."

Interviewer: "What happens if payment gateway times out?"

You: "We handle this with the SAGA pattern and circuit breaker. After 30 seconds, the circuit breaker opens and we return 'Processing your payment, we'll notify you'. Then a background job polls the payment gateway status API. Based on the response - success, failed, or still pending - we either confirm the booking, cancel it with refund, or retry. Every payment has an idempotency key to prevent double charges."

Interviewer: "How do you handle peak load during new movie releases?"

You: "Multi-pronged approach:"

You: "**1. Predictive Scaling** - We analyze historical data. Friday 6-10 PM = 10x normal load. We pre-scale booking service from 5 to 30 pods one hour before."

You: "**2. Rate Limiting** - 10 booking requests per user per minute, 500 per IP to prevent abuse."

You: "**3. Graceful Degradation** - Priority 1 is booking and payment. During high load, we disable nice-to-have features like recommendations and serve stale cache for search."

You: "**4. Queueing** - If bookings/sec exceeds threshold, we put users in a virtual queue showing their position."

You: "**5. Database Optimizations** - Read queries go to replicas, prepared statements, proper indexes on (show_id, seat_status, show_date)."

Interviewer: "How would you implement a waitlist feature?"

You: "I'd add a waitlist table with user_id, show_id, num_seats, and created_at timestamp. When a cancellation occurs, a Kafka event triggers the waitlist service. It queries the waitlist entries in FIFO order, checks if enough seats of the preferred type are available, and sends push notifications to the first matching users. They get 15 minutes to complete booking. If they don't, we cascade to the next person."

Interviewer: "How do you handle refunds?"

You: "Refunds follow business rules with automated processing. Here's the approach:"

You speaking:

```
REFUND WORKFLOW

Business Rules:
──────────────────────────────────────────────
> 24 hours before show:  100% refund (automatic)
4-24 hours before show:  50% refund (automatic)
< 4 hours before show:   0% refund (not allowed)

Technical Flow:
──────────────────────────────────────────────
1. Validation
   - Check booking status = CONFIRMED
   - Calculate time until show
   - Determine refund percentage

2. Database Transaction (ACID)
   BEGIN;
   UPDATE bookings SET status='CANCELLED';
   UPDATE show_seats SET status='AVAILABLE';
   INSERT INTO refunds (booking_id, amount, status);
   COMMIT;

3. Payment Gateway
   - Call gateway refund API
   - Use idempotency key: refund_{booking_number}
   - Prevents double refunds
```

```
4. Async Notification
   - Publish 'booking.cancelled' to Kafka
   - Email service sends confirmation
   - Push notification to user

5. Seat Release
   - Seats immediately available for rebooking
   - Redis cache invalidated
   - Waitlist notified if exists
```

You: "The idempotency key is critical - if the refund API call times out and we retry, the gateway recognizes it's a duplicate request and doesn't refund twice."

Interviewer: "What happens if Redis cluster goes down?"

You: "Great question - this is a critical failure scenario. Here's my defense-in-depth approach:"

You speaking:

```
REDIS FAILURE HANDLING

Scenario 1: Single Redis node fails
──────────────────────────────────────────────
- Redis Cluster with 3 primary + 3 replica nodes
- Automatic failover in <30 seconds
- Sentinel monitors and promotes replica
- Application reconnects automatically
→ Minimal impact, some locks may be lost but TTL ensures cleanup


Scenario 2: Entire Redis cluster down
──────────────────────────────────────────────
- Circuit breaker detects failures (3 failures in 10 sec)
- Circuit opens, Redis calls are bypassed
- Fallback: Use ONLY database locking (FOR UPDATE)
- Performance impact: Latency increases 200ms → 500ms
- Still functional but slower
→ Graceful degradation


Scenario 3: Network partition (split brain)
──────────────────────────────────────────────
- Redis Cluster uses majority quorum
- Minority partition rejects writes
- Database lock is the source of truth
- Double-booking prevented by DB constraint:
  UNIQUE(show_id, seat_number, status='BOOKED')


Recovery:
──────────────────────────────────────────────
```

```
- When Redis comes back online
- Sync state from database
- Rebuild available_seats cache from DB
- Resume normal operations
```

You: "The key insight is Redis is a performance optimization, not the source of truth. The database is authoritative. So if Redis completely fails, we fall back to database-only locking. It's slower but still correct."

Interviewer: "Design the APIs for the booking service."

You: "Let me show you the RESTful API design for the critical endpoints."

You speaking:

💡 **API DESIGN - MUST KNOW:**

```
BOOKING SERVICE APIs

1. Get Available Shows
──────────────────────────────────────────────
GET /api/v1/shows
Query Params:
  - city_id (required)
  - movie_id (optional)
  - date (optional, default: today)
  - theater_id (optional)

Response:
{
  "shows": [
    {
      "show_id": 12345,
      "movie": { "id": 1, "title": "Avengers", "duration": 180 },
      "theater": { "id": 10, "name": "PVR Phoenix" },
      "screen": { "id": 1, "name": "Screen 1", "type": "IMAX" },
      "show_time": "2024-08-15T19:30:00",
      "available_seats": 45,
      "pricing": {
        "REGULAR": 200,
        "PREMIUM": 300,
        "RECLINER": 500
      }
    }
  ]
}


2. Get Seat Layout
──────────────────────────────────────────────
GET /api/v1/shows/{showId}/seats
```

```
Response:
{
  "show_id": 12345,
  "total_seats": 120,
  "available_seats": 45,
  "layout": {
    "rows": [
      {
        "row_name": "A",
        "seats": [
          { "id": "A1", "type": "RECLINER", "status": "AVAILABLE", "price":
500 },
          { "id": "A2", "type": "RECLINER", "status": "LOCKED", "price":
500 },
          { "id": "A3", "type": "RECLINER", "status": "BOOKED", "price":
500 }
        ]
      }
    ]
  }
}


3. Lock Seats (Initiate Booking)
─────────────────────────────────────────────────
POST /api/v1/bookings/lock
Headers:
  Authorization: Bearer {jwt_token}

Body:
{
  "show_id": 12345,
  "seat_ids": ["A1", "A2"]
}

Response:
{
  "booking_id": "BMS20240815123456",
  "status": "PENDING",
  "expires_at": "2024-08-15T20:15:00",  // 10 min from now
  "expires_in_seconds": 600,
  "total_amount": 1000,
  "seats": [
    { "id": "A1", "price": 500 },
    { "id": "A2", "price": 500 }
  ]
}

Error Cases:
409 Conflict: "Seat A1 already selected by another user"
400 Bad Request: "Invalid seat_ids"
```

```
4. Confirm Booking (After Payment)
─────────────────────────────────────────────────
POST /api/v1/bookings/{bookingId}/confirm
Headers:
  Authorization: Bearer {jwt_token}

Body:
{
  "payment_id": "pay_xyz123",
  "payment_method": "UPI",
  "transaction_id": "TXN7384829"
}

Response:
{
  "booking_id": "BMS20240815123456",
  "status": "CONFIRMED",
  "booking_number": "BMS20240815123456",
  "qr_code": "base64_encoded_qr",
  "ticket_url": "https://cdn.bookmyshow.com/tickets/BMS20240815123456.pdf"
}


5. Cancel Booking
─────────────────────────────────────────────────
POST /api/v1/bookings/{bookingId}/cancel
Headers:
  Authorization: Bearer {jwt_token}

Response:
{
  "booking_id": "BMS20240815123456",
  "status": "CANCELLED",
  "refund_amount": 500,  // 50% refund
  "refund_status": "PENDING",
  "estimated_refund_days": "5-7 business days"
}


6. WebSocket - Real-time Seat Updates
─────────────────────────────────────────────────
WS /api/v1/shows/{showId}/seats/live

Client subscribes:
{
  "action": "subscribe",
  "show_id": 12345
}

Server pushes updates:
{
  "type": "seat_status_change",
  "seat_id": "A1",
  "status": "LOCKED",
```

```
  "timestamp": "2024-08-15T19:20:15"
}
```

You: "The WebSocket connection is crucial for real-time updates. When User B selects seat A1, all connected clients see it turn red immediately. This prevents multiple users from trying to book the same seat."

Interviewer: "How do you ensure payment security?"

You: "Payment security is critical. Here's my multi-layer approach:"

You speaking:

```
PAYMENT SECURITY

1. PCI-DSS Compliance
─────────────────────────────────────────────
- NEVER store credit card numbers
- Use tokenization (Stripe/Razorpay handles this)
- Store only last 4 digits + payment token
- All payment data encrypted in transit (TLS 1.3)


2. Payment Flow (Secure)
─────────────────────────────────────────────
Frontend:
  - User enters card details in Stripe iframe (not our form)
  - Stripe returns payment_token
  - Send token to our backend (not raw card data)

Backend:
  - Receive payment_token
  - Call Stripe API with token
  - Stripe charges the card
  - We get success/failure response
→ We NEVER see actual card details


3. Idempotency
─────────────────────────────────────────────
- Every payment has unique idempotency_key
- Format: {user_id}_{booking_id}_{timestamp}
- If network fails and we retry
- Gateway recognizes duplicate, doesn't double-charge


4. 3D Secure / OTP
─────────────────────────────────────────────
- For high-value transactions (>₹2000)
- Redirect to bank's OTP page
- Additional authentication layer
```

```
- Reduces fraud significantly


5. Fraud Detection
──────────────────────────────────────────────────
- Rate limiting: 5 payment attempts per user per hour
- Block suspicious patterns:
  - Same card on multiple accounts
  - Many failed attempts
  - Unusual booking patterns
- Integration with fraud detection services


6. Database Security
──────────────────────────────────────────────────
CREATE TABLE payments (
    id BIGSERIAL PRIMARY KEY,
    booking_id BIGINT,
    amount DECIMAL(10,2),
    payment_token VARCHAR(255),  -- Tokenized, not raw card
    last_4_digits VARCHAR(4),    -- Only last 4 for display
    status VARCHAR(20),
    -- NO card_number column!
    -- NO cvv column!
);
```

You: "The golden rule: if we get hacked, the attacker should find zero usable payment information. Everything is tokenized through payment gateways."

Interviewer: "How would you monitor this system?"

You: "Monitoring and observability are crucial. Let me break down my approach."

You speaking:

💡 **MONITORING STRATEGY:**

```
1. Service-Level Indicators (SLIs)
──────────────────────────────────────────────────
Booking Service:
- Success rate: > 99.9%
- Latency p50: < 200ms
- Latency p99: < 2s
- Lock acquisition time: < 10ms
- Lock failure rate: track for contention

Search Service:
- Latency p99: < 500ms
- Cache hit rate: > 80%
- Search relevance score
```

```
Payment Service:
- Payment success rate: > 99%
- Gateway timeout rate: < 1%
- Refund success rate: > 99.5%


2. Business Metrics (Real-time Dashboard)
──────────────────────────────────────────────────
- Bookings per minute (by city, theater, movie)
- Revenue per minute
- Average booking value
- Conversion rate (searches → bookings)
- Seat utilization (booked/total)
- Cancellation rate
- Top performing movies/theaters


3. Infrastructure Metrics
──────────────────────────────────────────────────
- CPU/Memory/Disk utilization per service
- Redis: Hit rate, memory usage, eviction rate
- PostgreSQL: Connection pool, query latency, replication lag
- Kafka: Consumer lag, message throughput
- Kubernetes: Pod count, auto-scaling events


4. Distributed Tracing (Jaeger/AWS X-Ray)
──────────────────────────────────────────────────
Example trace for booking:

Trace ID: abc123
Total: 450ms

├─ API Gateway: 10ms
├─ Auth validation: 20ms
├─ Booking Service: 380ms
│  ├─ Redis lock acquire: 5ms
│  ├─ Database query: 120ms ← SLOW! Needs optimization
│  ├─ Payment service call: 240ms
│  └─ Kafka publish: 15ms
└─ Response serialization: 40ms

→ Identifies that DB query is the bottleneck


5. Alerting (PagerDuty / Opsgenie)
──────────────────────────────────────────────────
Critical (Page on-call engineer):
- Booking service error rate > 1%
- Payment gateway down
- Database connection pool exhausted
- Redis cluster down
- P99 latency > 5s
```

```
Warning (Slack notification):
- Cache hit rate < 70%
- Pending bookings > 1000
- Disk usage > 80%
- Unusual cancellation spike


6. Logging (ELK Stack)
──────────────────────────────────────────────
Structured logging format:
{
  "timestamp": "2024-08-15T19:30:45Z",
  "level": "ERROR",
  "service": "booking-service",
  "trace_id": "abc123",
  "user_id": 456,
  "booking_id": "BMS20240815123456",
  "error": "Lock acquisition failed",
  "seat_ids": ["A1", "A2"],
  "show_id": 12345
}

→ Enables quick debugging with trace_id


7. Health Checks
──────────────────────────────────────────────
GET /health/live   (Kubernetes liveness)
GET /health/ready  (Kubernetes readiness)

Response:
{
  "status": "healthy",
  "checks": {
    "database": "UP",
    "redis": "UP",
    "kafka": "UP"
  },
  "uptime_seconds": 86400
}
```

You: "The key is having different alert thresholds for different metrics. Not everything deserves a 3 AM page. Critical path failures like 'can't book tickets' are P0. Cache hit rate drop is P3."

SECTION 8: TRADE-OFFS & DESIGN DECISIONS

Phase 7: Deep Discussion (3 minutes)

You: "Let me explain some key trade-offs I made in this design."

You speaking:

```
CRITICAL TRADE-OFFS

1. Redis + Database Locking (Not just one)
─────────────────────────────────────────────────
Considered:
  A) Only Redis locking
  B) Only Database locking
  C) Both (chosen)

Chose C because:
  ✓ Redis: Fast (10ms), handles 99% of conflicts
  ✓ Database: ACID guarantee, handles edge cases
  ✗ Complexity: Need to maintain both
  ✗ Cost: Additional infrastructure

Why not A? Redis isn't ACID-compliant, risky for money
Why not B? Database locking doesn't scale to 10K concurrent users


2. PostgreSQL vs NoSQL for Bookings
─────────────────────────────────────────────────
Considered:
  A) PostgreSQL (chosen)
  B) MongoDB
  C) DynamoDB

Chose A because:
  ✓ ACID transactions critical for money
  ✓ Relational data (bookings ↔ seats ↔ payments)
  ✓ Complex joins needed
  ✓ Strong consistency required

Why not B/C? Eventual consistency unacceptable for bookings


3. Microservices vs Monolith
─────────────────────────────────────────────────
Considered:
  A) Monolith
  B) Microservices (chosen)

Chose B because:
  ✓ Different scaling needs (search ≠ booking)
  ✓ Team autonomy (search team ≠ payment team)
  ✓ Technology flexibility
  ✗ Distributed system complexity
  ✗ Network latency between services

Worth it? Yes, because scale demands it


4. Sync vs Async for Notifications
```

```
─────────────────────────────────────────────────
Considered:
  A) Synchronous email/SMS in booking flow
  B) Asynchronous via Kafka (chosen)

Chose B because:
  ✓ Booking completes faster (don't wait for email)
  ✓ Retry mechanism if email fails
  ✓ Decoupling (booking ≠ notification)
  ✗ User might not get immediate email

Trade-off accepted: Slight delay in email OK


5. Cache TTL: Freshness vs Performance
─────────────────────────────────────────────────
Search results: 5 minutes
  - Movies/theaters don't change often
  - Stale data acceptable

Seat availability: 30 seconds
  - Needs to be relatively fresh
  - But not real-time (WebSocket for that)

Booking data: No cache
  - Always fetch from DB
  - Money involved, can't risk stale data


6. Sharding Strategy
─────────────────────────────────────────────────
Considered:
  A) By user_id (hash)
  B) By city_id (chosen)
  C) By date range

Chose B because:
  ✓ 95% queries filtered by city
  ✓ Geographic data locality
  ✓ Hot shard isolation (Mumbai)

Why not A? Users book in different cities
Why not C? Date ranges change, complex to maintain
```

You: "Every design decision is a trade-off. The key is understanding WHAT you're trading and WHY it's worth it."

APPENDIX: QUICK REFERENCE

💡 **KEY NUMBERS TO REMEMBER:**

```
- 100M users, 10M bookings/month
- Peak: 92 bookings/sec
- Lock TTL: 10 minutes
- Payment timeout: 30 seconds
- Cache TTL: 5 min (search), 30 sec (seats)
- Read:Write ratio: 100:1
```

💡 **CRITICAL COMPONENTS:**

```
- Redis SETNX for distributed locking
- PostgreSQL FOR UPDATE for database locking
- Kafka for event streaming
- Elasticsearch for geo + full-text search
- MongoDB for flexible theater layouts
- Circuit breaker for payment gateway
```

💡 **MUST-MENTION PATTERNS:**

```
✓ Distributed locking (Redis + DB)
✓ SAGA pattern (payment failures)
✓ CQRS (read/write separation)
✓ Event sourcing (Kafka)
✓ Sharding by city
✓ Multi-level caching
✓ Auto-scaling (Kubernetes HPA)
✓ Rate limiting (token bucket)
```

END OF INTERVIEW GUIDE

This guide covers all the critical aspects of BookMyShow system design with natural conversational flow and clear indicators of what to write/draw in the interview.